[Project Name]

Senior Design (423-IDE401 / 424-IDE402) Core Report

Version 1.0

Draft Template

by

[Senior Design Team]

Stevens Institute of Technology

[Month Day, Year]

© Stevens Institute of Technology 2026
ALL RIGHTS RESERVED

Template Attribution

This Senior Design Core Report template was adapted from the software architecture design template developed by **Darian Muresan, Ph.D.**, in conjunction with the AI agents **OpenAI Codex** and **Anthropic Claude Code**.

Contact: dmuresan@stevens.edu
darian@dmmd.net
darian@pictomic.com

[Project Name]

Senior Design (423-IDE401 / 424-IDE402) Core Report

Version 1.0
Draft Template

[Senior Design Team]
Stevens Institute of Technology
[Month Day, Year]

This document specifies the stakeholder needs, requirements, architecture, implementation constraints, tests, verification evidence, validation evidence, and release criteria for the [Project Name]. It is both a printable engineering record and the authoritative input for AI-assisted implementation. A conforming implementation may be regenerated from the complete specification, but no single chapter should be treated as sufficient on its own.

Document History

Add the newest approved change at the top of the table. Retain every approved entry so that the report's evolution and review history remain auditable.

Table 2: Document Update History

Date / Version	**Author**	**Change**	**Location**
[date / version]	*[initials]*	*[approved change and rationale]*	*[section]*
September 1, 2026 / 1.0	DDM, CDX	Adapted the system and software architecture design template as the Senior Design Core Report template. Added the complete 423-IDE401 / 424-IDE402 report structure, milestones, and appendices.	Entire report
[baseline date / 0.1]	*[initials]*	*[project-specific baseline]*	*[section]*

Table of Contents

Template Attribution iii

Document History v

1 Introduction 1
- 1.1 How to Use This Chapter . . . 1
- 1.2 Purpose . . . 1
- 1.3 Scope . . . 1
- 1.4 Audience . . . 2
- 1.5 Normative Language and Priority . . . 2
- 1.6 Document as Source of Truth . . . 2
- 1.7 Using This Template with AI Agents . . . 2
- 1.8 V-Model Organization . . . 3

Executive Summary 5

I Project Definition and Plan 6

2 Mission Statement 7

3 Background 8

4 Stakeholder List 9

5 Analysis of Stakeholder Needs 10

6 Project Scope and Resources 11

7 Project Plan 12

II Design, Evaluation, and Optimization **13**

8 Requirements **14**
8.1 How to Use This Chapter . . . 14
8.2 Functional and Quality Requirements . . . 14
8.3 Requirement Status Model . . . 15
8.4 Requirement Category Catalog . . . 15
8.5 Creating a Project-Specific Category . . . 19
8.6 Review Rules . . . 19
8.7 Category-Specific Requirement Registers . . . 20
8.7.1 Business Requirements . . . 20
8.7.2 Functional Requirements . . . 21
8.7.3 Quality Requirements . . . 21
8.7.4 Interface Requirements . . . 22
8.7.5 Security Requirements . . . 22
8.7.6 Data Requirements . . . 23
8.7.7 Constraint Requirements . . . 23

9 User Stories **24**
9.1 How to Use This Chapter . . . 24
9.2 User Story Register . . . 24
9.3 Review Rules . . . 25

10 Use Cases **26**
10.1 How to Use This Chapter . . . 26
10.2 High-Level Use Case Diagram . . . 26
10.3 Submit a Service Request . . . 26
10.4 View Request Status . . . 27
10.5 Export Request History . . . 27

11 Architecture Overview **29**
11.1 How to Use This Chapter . . . 29
11.2 Decision Template and Example . . . 29

12 User Interface Design **30**
12.1 How to Use This Chapter . . . 30
12.2 Example . . . 30

13 Logical View **31**
13.1 How to Use This Chapter . . . 31
13.2 Example . . . 31

14 Process View **32**
14.1 How to Use This Chapter . . . 32
14.2 Example State Model . . . 32

15 Development View **33**
15.1 How to Use This Chapter . . . 33
15.2 Example . . . 33

16 Physical View **34**
16.1 How to Use This Chapter . . . 34
16.2 Example . . . 34

17 Constraints and Assumptions **35**
17.1 Design and Implementation Constraints . . . 35
17.2 Assumptions and Dependencies . . . 35

18 Applicable Codes, Standards, and Regulations **36**

19 Concept Development and Selection **37**

20 Preliminary and Detailed Design **38**
20.1 System Architecture . . . 38
20.1.1 Overview of System Architecture . . . 38
20.1.2 Hardware Architecture . . . 38
20.1.3 Software Architecture . . . 38
20.1.4 Security Architecture . . . 38
20.1.5 Communication Architecture . . . 38
20.2 System Design . . . 39
20.2.1 System Behavior Description . . . 39
20.2.2 Databases/Data Storage . . . 39
20.2.3 Data Conversion . . . 39
20.2.4 Sub-System Interfaces . . . 39
20.2.5 User Interface Design . . . 39
20.3 Mechanical Design . . . 39
20.3.1 Components and Sub-Assemblies - Bill of Materials . . . 39
20.3.2 Geometric Specification - CAD Models/Drawings . . . 39
20.3.3 Materials Selected . . . 39
20.3.4 Manufacturing Processes Selected . . . 39
20.3.5 Process Description or Optimization Notes . . . 40
20.4 Electronics/Electro-Mechanical Design . . . 40
20.4.1 Circuits . . . 40
20.4.2 Electrical and Electro-Mechanical Interfaces . . . 40
20.4.3 List of Components: Motors, Sensors, Electronics . . . 40
20.4.4 Power System . . . 40
20.5 Software Components Design . . . 40
20.5.1 Algorithms . . . 40
20.5.2 Modules/Classes/Methods . . . 40
20.5.3 Application Programming Interfaces . . . 40
20.5.4 Coding/Implementation Details . . . 40

21 Design Evaluation Methods **41**
21.1 Design Concepts and Concept Screening . . . 41
21.2 Design Evaluation/Selection Methods . . . 41
21.3 Analysis Cases and Results . . . 41
21.4 Impact on System Design . . . 41

22 Alpha Prototyping: Plan and Budget **42**

23 Testing and Evaluation: Plan and Results **43**

24 Tests **44**
24.1 How to Use This Chapter . . . 44
24.2 Example Test Cases . . . 44

25 Traceability **46**
25.1 How to Use This Chapter . . . 46
25.2 Traceability Register . . . 46

26 Verification **48**
26.1 How to Use This Chapter . . . 48
26.2 Verification Register . . . 48

27 Validation **50**
27.1 How to Use This Chapter . . . 50
27.2 Validation Register . . . 50

28 Beta Prototyping: Design Revision and Optimization **53**
28.1 Mechanical Design . . . 53
28.2 Electronics/Electro-Mechanical Design . . . 53
28.3 Software Design . . . 53
28.4 System-Level Design . . . 53

29 Fabrication, Assembly, and Manufacturing Processes **54**

30 High-Performance Testing and Evaluation **55**

31 Final Design Specification **56**

III Entrepreneurship and Business Development **57**

32 Business Objectives and Risks **58**

33 Competitive Intelligence: Market Analysis **59**

34 Lean Canvas Business Model **60**

35 Financial Analysis **61**

36 Intellectual Property **62**

37 Innovation Expo **63**

IV Appendices **64**

A Team Organization Chart **65**

B Project Gantt Chart **66**

C Prototyping and Testing Budget **67**

D Design Documents: Drawings, Layouts, Analysis Reports **68**

E References Cited **69**

F Document Authoring Guide **70**
F.1 How to Use This Appendix . . . 70
F.2 Template Provenance . . . 70
F.3 AI-Assisted Authoring . . . 70
F.4 File Naming . . . 70
F.5 One Responsibility per Chapter . . . 71
F.6 Adding a Chapter . . . 71
F.7 Building the PDF . . . 71
F.7.1 Installing the Tools on macOS . . . 71
F.7.2 Installing the Tools on Windows . . . 72
F.7.3 Installing the Tools on Ubuntu or Debian Linux . . . 73
F.7.4 AI-Agent Installation . . . 73
F.7.5 Building on macOS and Linux . . . 73
F.7.6 Building with Windows PowerShell . . . 74
F.7.7 Manual Build Sequence . . . 74
F.7.8 Build Verification and Troubleshooting . . . 74
F.8 Authoring Rules . . . 75
F.9 Citations and Bibliography . . . 75
F.10 Selective Builds . . . 75

G AI Agent Usage Guide **76**
G.1 How to Use This Appendix . . . 76
G.2 Governing Principles . . . 76
G.3 AI Quick Start . . . 76
G.4 Task Routing . . . 77
G.5 Task-Specific Context Packs . . . 78
G.5.1 Requirements Context . . . 78

G.5.2 Use-Case Context . . . 78
G.5.3 Architecture Context . . . 78
G.5.4 Testing and Evidence Context . . . 79
G.5.5 Release Context . . . 79
G.5.6 Document-Authoring Context . . . 79
G.6 Mandatory Consistency Rules . . . 79
G.7 Standard AI Workflows . . . 80
G.7.1 Initialize a Project . . . 80
G.7.2 Add or Change a Requirement . . . 80
G.7.3 Add or Change a Use Case . . . 80
G.7.4 Develop or Change Architecture . . . 80
G.7.5 Add Tests or Evidence . . . 80
G.7.6 Review and Prepare a Release . . . 80
G.8 Example AI Requests and Prompts . . . 81
G.8.1 Initialize the Architecture Book . . . 81
G.8.2 Develop Requirements . . . 81
G.8.3 Develop Use Cases . . . 81
G.8.4 Develop an Architecture Decision . . . 82
G.8.5 Develop Tests and Traceability . . . 82
G.8.6 Review Release Readiness . . . 82
G.9 Human Decisions and Information Boundaries . . . 82
G.10 Completion Checklist . . . 82

H AI Prompt Log **84**
H.1 How to Use This Appendix . . . 84
H.2 Example Record . . . 84

Bibliography **85**

Glossary **86**

Acronyms **87**

Index **88**

List of Tables

2 Document Update History v

8.1 Requirement Status Definitions 15
8.2 Requirement Families, Subtypes, and Quality Attributes 16
8.3 Business Requirements Register 20
8.4 Functional Requirements Register 21
8.5 Quality Requirements Register 21
8.6 Interface Requirements Register 22
8.7 Security Requirements Register 22
8.8 Data Requirements Register 23
8.9 Constraint Requirements Register 23

9.1 User Story Register 24

24.1 Test Case Examples 44

25.1 Traceability Register 47

26.1 Verification Register 48

27.1 Validation Register 50

G.1 AI Task-Routing Table 77
G.2 AI-Assisted Change Completion Checklist 83

List of Figures

1.1 V-model relationships used by this specification. 4

10.1 Generic high-level use case diagram. 26

Chapter 1

Introduction

1.1 How to Use This Chapter

State the document purpose, scope, audience, conventions, and relationship to other project artifacts. Replace bracketed text with project-specific content. Keep binding obligations in the Requirements chapter.

1.2 Purpose

This document is the authoritative engineering specification for [Project Name]. It records why the system is needed, what it shall do, how it is structured, how it will be tested, and what evidence is required before release. It is readable by people and structured for use by engineering and artificial intelligence (AI)-assisted development tools.

Central lesson

A vague request produces unpredictable software. A precise, traceable specification allows engineers to generate, review, test, and, when necessary, regenerate a conforming implementation.

1.3 Scope

Define the system boundary, included capabilities, exclusions, operating environment, major constraints, and the release covered by this document.

Generic example: [Project Name] enables [primary user] to [achieve outcome]. This release includes [included capabilities] and excludes [out-of-scope capabilities]. It operates in [target environment] and depends on [external systems or services].

1.4 Audience

Identify the people who create, approve, implement, test, operate, maintain, audit, or use the system. State which sections are most relevant to each role.

Generic example: The primary audiences are customer representatives, systems engineers, architects, developers, testers, operators, maintainers, security reviewers, and project approvers. One person may perform several roles on a small project.

1.5 Normative Language and Priority

Use **shall** for every binding requirement. Do not use **should** as guidance in normative statements. Assign each requirement one priority: **Must**, **Should**, **Could**, or **Would**. Priority expresses delivery importance; it does not change whether a statement containing **shall** is binding for its assigned implementation version.

1.6 Document as Source of Truth

Requirements alone do not define a reproducible implementation. The complete source of truth consists of stakeholder needs, user stories, requirements, use cases, architecture and interface decisions, implementation constraints, tests, verification evidence, validation evidence, and release criteria. Generated code is an implementation artifact governed by this specification.

1.7 Using This Template with AI Agents

AI agents such as OpenAI Codex and Anthropic Claude Code can use the LaTeX sources as structured project context. The agent shall begin with Chapter G, select the applicable task-routing entry, and read only the authoritative files identified for that task. It shall expand the context only when a referenced dependency or unresolved inconsistency requires it. This keeps routine work focused without weakening traceability.

The AI Agent Usage Guide contains the task-routing table, context packs, mandatory rules, standard workflows, completion checklists, human-review boundaries, and reusable prompts. Record material AI-assisted decisions in Chapter H.

The following minimal request starts an AI-assisted task:

```
Read dsnAIAgentGuide.tex first. Identify the applicable task and
load only the files listed for it. Follow the template's stable-ID,
priority, status, traceability, history, build, and review rules.
Use only supplied project evidence. Report missing information and
decisions that require human approval. Complete the requested task,
compile the document, and visually inspect the affected PDF pages.
```

1.8 V-Model Organization

The left side of the V-model moves from stakeholder needs toward detailed design. The bottom represents implementation. The right side integrates the system and gathers evidence, pairing acceptance with stakeholder needs, system verification with requirements, integration tests with architecture, and unit tests with detailed design.

This organization is compatible with the system and software life-cycle process frameworks in [1, 2]; those standards do not require a particular life-cycle model.

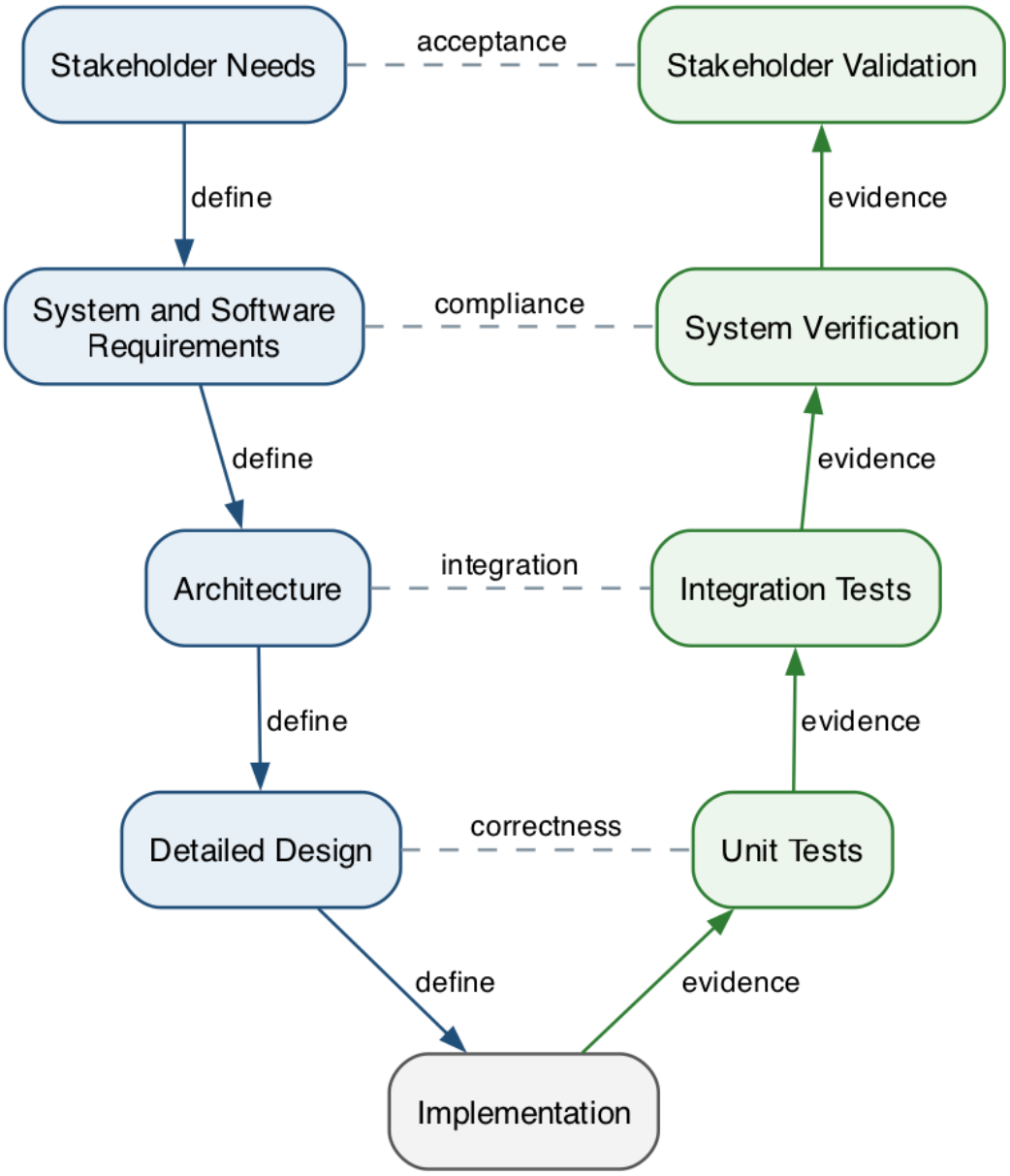


Figure 1.1: V-model relationships used by this specification.

Executive Summary

Summarize the problem, the proposed solution, major design decisions, prototype status, evaluation results, and expected impact. Keep this section readable by sponsors and reviewers who may not read the full report.

Part I

Project Definition and Plan

Chapter 2

Mission Statement

State the project mission, the intended benefit, and the measurable outcome the team will deliver.

Chapter 3

Background

Explain the context, prior work, evidence, and motivating problem. Cite supporting sources in Appendix E and the bibliography.

Chapter 4

Stakeholder List

Identify users, sponsors, customers, operators, maintainers, regulators, and other affected parties. State each stakeholder's role and interest.

Chapter 5

Analysis of Stakeholder Needs

Document stakeholder needs, priorities, conflicts, and the evidence used to understand them. Link needs to requirements in Chapter 8.

Chapter 6

Project Scope and Resources

Define in-scope and out-of-scope work, team capabilities, facilities, tools, funding, and other resources required for delivery.

Chapter 7

Project Plan

Describe the work breakdown, responsibilities, schedule, risk management, review gates, and change-control approach. Include the team organization chart in Appendix A and the Gantt chart in Appendix B.

Part II

Design, Evaluation, and Optimization

Chapter 8

Requirements

8.1 How to Use This Chapter

State singular, necessary, unambiguous, feasible, and verifiable obligations. Every requirement shall have a stable ID, one priority, the version in which it was implemented, at least one mapped use case, and at least one mapped test case. Enter ∅ in **In Version** when no implementation version has been assigned. Preserve IDs when wording or version assignments change.

Select the requirement categories that make the specification easier to review and govern. Keep each requirement in one primary register and use traceability links instead of copying the same requirement into multiple categories.

8.2 Functional and Quality Requirements

Functional and quality requirements answer different questions and shall not be used interchangeably.

Central Distinction

A **functional requirement** states what behavior, service, calculation, state transition, or externally observable result the system shall provide. A **quality requirement** states how well that behavior or the system as a whole shall perform under stated conditions. Quality requirements require a measurable threshold, workload, environment, or other evaluation basis.

For example, "The system shall display the current request status" is functional. "The system shall display the current request status within two seconds under the reference workload" is a quality requirement. Avoid vague quality statements such as "the system shall be fast, secure, reliable, or easy to use." Replace them with measurable conditions and success criteria.

Quality requirements are also called quality-attribute or non-functional requirements. The term *quality requirement* is preferred here because these obligations describe behavior

that shall be designed, implemented, and verified rather than an absence of function.

8.3 Requirement Status Model

Priority and status are independent. Priority records the requirement's importance or commitment; status records its progress through the delivery lifecycle. A status change shall not erase or replace the assigned priority.

Table 8.1: Requirement Status Definitions

Status	Meaning	In Version Rule
Open	The requirement is active, but implementation has not started. Deferred work remains Open until work begins or the requirement is retired.	Enter ∅.
In Progress	Implementation or integration work has started, but the requirement does not yet satisfy the definition of Done.	Enter ∅.
Done	The requirement is implemented; mapped tests pass; verification is complete; traceability is current; and the completed behavior is included in a controlled version.	Enter the first controlled version containing the completed requirement.
Retired	The requirement is no longer active because it was superseded, made obsolete, rejected, or withdrawn. Retain its stable ID, original text, and traceability; record the rationale and any replacement in document history.	Retain the original implementation version if it was previously Done; otherwise enter ∅.

Only **Done** asserts completion. Do not remove a retired requirement or reuse its ID. A requirement may move from **Retired** only through an approved change that restores its active status and records the decision in document history.

8.4 Requirement Category Catalog

The following catalog distinguishes broad requirement families from their subtypes. In particular, performance, capacity, scalability, reliability, availability, resilience, recoverability, usability, accessibility, maintainability, supportability, testability, portability, compatibility, interoperability, security, privacy, safety, sustainability, and observability are quality attributes. A requirement concerning one of these attributes is a quality requirement when it specifies how well the system shall operate under stated conditions.

A project may organize a specialized quality attribute, such as security, privacy, or safety,

in a separate register when it has distinct owners, review criteria, evidence, or assurance obligations. The separate register is a management view; it does not change the requirement's classification as a quality requirement. Likewise, interface, data, operational, and other families may contain both functional obligations and associated quality obligations. Use the primary purpose of each requirement to select its register and use links to represent cross-cutting concerns.

The catalog is intentionally broad. A project may use a family as a top-level register, create registers for selected subtypes, combine closely related groups, or omit groups that do not apply. Category choice shall not weaken the traceability or verification rules for individual requirements.

Table 8.2: Requirement Families, Subtypes, and Quality Attributes

Requirement Family	**Subtype or Quality Attribute**	**Typical Scope**
Business	Outcomes and policy	Organizational outcomes, value, policy goals, measurable benefits, cost targets, and mission objectives.
Stakeholder and user	Stakeholder need	Capabilities, outcomes, and constraints needed by a stakeholder group or external party.
Stakeholder and user	User need	User goals and externally visible outcomes expressed independently of a particular implementation.
Functional	System behavior	Services, calculations, decisions, state changes, workflows, and observable system responses.
Quality	General quality	Measurable characteristics that state how well a function or the system shall operate under stated conditions.
Quality	Performance	Response time, latency, throughput, processing time, and resource consumption.
Quality	Capacity	Supported data volume, transaction volume, concurrent users, devices, or retained records.
Quality	Scalability	Behavior as workload, data, users, or deployed instances increase or decrease.
Quality	Reliability	Probability or frequency of failure-free operation under stated conditions.
Quality	Availability	Required service availability, planned downtime, and permitted outage duration.
Quality	Resilience and fault tolerance	Continued or degraded operation when components, dependencies, or communication paths fail.
Quality	Recoverability and continuity	Recovery time, recovery point, restoration, backup, failover, and disaster-continuity objectives.

Continued on next page

Requirement Families, Subtypes, and Quality Attributes (continued)

Requirement Family	Subtype or Quality Attribute	Typical Scope
Quality	Usability	Task effectiveness, efficiency, learnability, error prevention, and user satisfaction.
Quality	Accessibility	Conformance and measurable support for users with visual, auditory, motor, speech, or cognitive disabilities.
Quality	Maintainability	Time, effort, modularity, analyzability, and change characteristics required for system modification.
Quality	Supportability	Diagnostic access, service procedures, field replacement, support hours, and support-team capabilities.
Quality	Testability	Controllability, observability, repeatability, test interfaces, and evidence needed to verify behavior.
Quality	Portability	Ability to install, move, or operate the system across specified environments or platforms.
Quality	Compatibility	Coexistence and correct operation with specified versions, devices, browsers, operating systems, or products.
Quality	Interoperability	Coordinated behavior, protocols, semantics, orchestration, and exchanges with external systems.
Quality	Security	Authentication, authorization, confidentiality, integrity, cryptography, auditing, and threat resistance.
Quality	Privacy	Collection, consent, purpose, minimization, access, disclosure, retention, deletion, and rights for personal data.
Quality	Safety	Hazard prevention, risk reduction, safe state, interlocks, alarms, and unacceptable-loss controls.
Quality	Sustainability	Energy use, resource efficiency, carbon reporting, hardware life, repair, reuse, and disposal objectives.
Quality	Observability	Logs, metrics, traces, health signals, diagnostic context, and measurable diagnostic coverage.
Interface	User interface	Human-system interaction, presentation, navigation, input, feedback, and interaction contracts.
Interface	Software and API	Operations, messages, schemas, errors, protocols, sequencing, and software boundary contracts.
Interface	Hardware and communication	Connectors, signals, electrical characteristics, device protocols, networks, and physical boundaries.

Continued on next page

Requirement Families, Subtypes, and Quality Attributes (continued)

Requirement Family	**Subtype or Quality Attribute**	**Typical Scope**
Data and information	Data definition and quality	Data definitions, schemas, provenance, accuracy, completeness, integrity, ownership, and permitted transformations.
Data and information	Lifecycle and governance	Creation, access, retention, archival, legal hold, disposition, deletion, and proof of destruction.
Operational	Operation and administration	Startup, shutdown, scheduling, configuration, provisioning, monitoring, backup, recovery, and incident response.
Operational	Deployment and platform	Approved platforms, topology, packaging, installation, configuration, upgrade, and runtime environment.
Operational	Infrastructure and environment	Network, compute, storage, facility, power, temperature, physical location, and environmental limits.
Transition	Migration and rollout	Data conversion, coexistence, rollout, cutover, rollback, legacy integration, and decommissioning.
Compliance and legal	Regulatory and standards	Obligations imposed by regulations, standards, certifications, policies, or assurance frameworks.
Compliance and legal	Legal and contractual	Licenses, warranties, intellectual-property terms, service agreements, procurement terms, and contractual duties.
Compliance and legal	Reporting and audit	Reports, evidence, audit trails, review access, content, timing, and record accountability.
Product support	Documentation	Required user, operator, administrator, developer, API, maintenance, and compliance documentation.
Product support	Training	Required curricula, exercises, qualifications, delivery modes, and demonstrated learner outcomes.
Commercial	Licensing and entitlement	Product editions, entitlement, metering, third-party components, subscriptions, and licensing restrictions.
AI and machine learning	Model and data controls	Model performance, data provenance, bias evaluation, explainability, human oversight, drift, and retraining controls.
Physical and hardware	Physical characteristics	Dimensions, mass, materials, connectors, tolerances, environmental ratings, and hardware behavior.
Constraint	Solution constraint	Mandated technologies, designs, budgets, schedules, standards, dependencies, or solution boundaries.

8.5 Creating a Project-Specific Category

Create an additional category when it gives requirements a distinct meaning, owner, review method, or assurance process. Define the category's scope and a unique ID prefix; duplicate one of the category-specific registers below; keep the standard columns; add the category to the category catalog; and update use cases, tests, traceability, verification, validation, document history, and the index as applicable. Document whether the new category is top-level or a subtype of an existing category.

Do not create a category solely to avoid resolving an ambiguous requirement. If one statement contains multiple independently verifiable obligations, split it into singular requirements and assign each one a primary category.

8.6 Review Rules

Reject a requirement row if its priority, status, use-case mapping, or test-case mapping is empty. The **Status** field shall contain Open, In Progress, Done, or Retired. The **In Version** field shall follow the status definitions. Confirm that reciprocal mappings appear in the user-story, use-case, test, traceability, and verification registers.

8.7 Category-Specific Requirement Registers

The seven registers below are the template's default working set. They are selected for practical review and ownership, not as seven mutually exclusive top-level families. For example, the Security Requirements Register is a specialized quality-requirement register. Add, rename, combine, or remove registers to match the approved project taxonomy. Every retained register shall use the same traceability columns.

8.7.1 Business Requirements

Table 8.3: Business Requirements Register

ID	Requirement	Priority	Status	In Version	Use Case(s)	Test Case(s)
REQ-B-001	The system shall enable an authorized analyst to prepare approved request-history data for analysis within five minutes for the reference data set.	Must	Open	∅	UC-003	TC-009

8.7.2 Functional Requirements

Table 8.4: Functional Requirements Register

ID	Requirement	Priority	Status	In Version	Use Case(s)	Test Case(s)
REQ-F-001	The system shall accept a valid service request and assign it a unique identifier.	Must	Done	1.0	UC-001	TC-001
REQ-F-002	The system shall display the current status and most recent update for a submitted request.	Should	Done	1.1	UC-002	TC-002
REQ-F-003	The system shall export request history in the approved interchange format.	Would	Open	∅	UC-003	TC-004

8.7.3 Quality Requirements

Table 8.5: Quality Requirements Register

ID	Requirement	Priority	Status	In Version	Use Case(s)	Test Case(s)
REQ-Q-001	The system shall display a request status within two seconds under the reference workload.	Could	Done	1.2	UC-002	TC-003

8.7.4 Interface Requirements

Table 8.6: Interface Requirements Register

ID	Requirement	Priority	Status	In Version	Use Case(s)	Test Case(s)
REQ-I-001	The system shall exchange request-history exports using the approved, versioned interface schema.	Should	**In Progress**	∅	UC-003	TC-005

8.7.5 Security Requirements

Table 8.7: Security Requirements Register

ID	Requirement	Priority	Status	In Version	Use Case(s)	Test Case(s)
REQ-S-001	The system shall permit request-history exports only to authenticated actors assigned the approved export role.	Must	**Done**	1.0	UC-003	TC-006

[Project Name] © Stevens Institute of Technology Version 1.0

8.7.6 Data Requirements

Table 8.8: Data Requirements Register

ID	Requirement	Priority	Status	In Version	Use Case(s)	Test Case(s)
REQ-D-001	The system shall retain each request status transition with its timestamp, actor identifier, previous status, and new status.	Must	Done	1.1	UC-002, UC-003	TC-007

8.7.7 Constraint Requirements

Table 8.9: Constraint Requirements Register

ID	Requirement	Priority	Status	In Version	Use Case(s)	Test Case(s)
REQ-C-001	The system shall support the deployment platform and browser versions identified in the approved deployment baseline.	Must	Done	1.0	UC-001, UC-002, UC-003	TC-008

Chapter 9

User Stories

9.1 How to Use This Chapter

Capture stakeholder goals in accessible language. Stories express value and acceptance intent; precise obligations belong in Requirements. Give each story a stable ID, a priority, and links to one or more requirements that implement it.

9.2 User Story Register

Table 9.1: User Story Register

ID	Story and Acceptance Criteria	Priority	Requirement(s)
US-001	As a customer, I want to submit a service request so that the responsible team can act on it. Acceptance: valid data produces a unique request identifier on every supported client.	Must	REQ-F-001, REQ-C-001
US-002	As a customer, I want to view my request status so that I understand its progress. Acceptance: the current status and most recent update appear within the specified response time, and the transition history is retained.	Should	REQ-F-002, REQ-Q-001, REQ-D-001, REQ-C-001
US-003	As an operations analyst, I want to export request history so that I can perform approved analysis. Acceptance: an authorized actor prepares complete history within five minutes in the approved versioned format without changing source records.	Would	REQ-B-001, REQ-F-003, REQ-I-001, REQ-S-001, REQ-D-001, REQ-C-001

9.3 Review Rules

Confirm that every story identifies a beneficiary, outcome, acceptance criterion, priority, and at least one linked requirement. Maintain reciprocal story links in the traceability register.

Chapter 10

Use Cases

10.1 How to Use This Chapter

Describe goal-oriented interactions at the system boundary. Give every use case a stable ID, its own table, and at least one reciprocal requirement mapping. Include preconditions, trigger, main flow, alternate or exception flows, and postconditions. Maintain the canonical relationships among stories, requirements, use cases, and tests in Chapter 25.

10.2 High-Level Use Case Diagram

Show the system boundary, primary actors, and major user goals before the detailed use-case tables. Replace the generic example with project-specific actors and use cases while preserving the stable use-case identifiers.

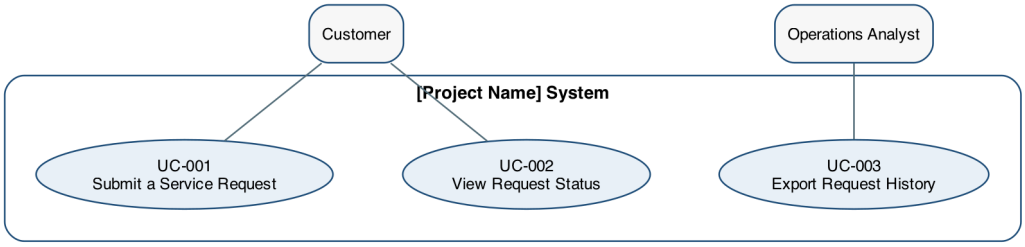


Figure 10.1: Generic high-level use case diagram.

10.3 Submit a Service Request

Identifier	`UC-001`

Mapped requirements	REQ-F-001, REQ-C-001
Goal	Submit a valid request and receive a unique identifier.
Primary actor	Customer
Preconditions	The customer can access the request form.
Trigger	The customer chooses to submit a new request.
Main flow	The customer enters required data and submits it; the system validates the data, creates the request, and displays its identifier.
Alternate flow	If required data is invalid, the system identifies the affected fields and creates no request.
Postconditions	A valid request exists with a unique identifier and an initial status.

10.4 View Request Status

Identifier	UC-002
Mapped requirements	REQ-F-002, REQ-Q-001, REQ-D-001, REQ-C-001
Goal	Review the current state of a submitted request.
Primary actor	Customer
Preconditions	A request with a known identifier exists.
Trigger	The customer opens or refreshes the request status.
Main flow	The system retrieves and displays the request status and most recent update.
Alternate flow	If the identifier is unknown, the system reports that no matching request was found.
Postconditions	The request remains unchanged; its current status has been presented.

10.5 Export Request History

Identifier	UC-003
Mapped requirements	REQ-B-001, REQ-F-003, REQ-I-001, REQ-S-001, REQ-D-001, REQ-C-001
Goal	Export request history for authorized analysis.
Primary actor	Operations analyst
Preconditions	The actor is authorized and matching requests exist.
Trigger	The actor selects an export range and format.

Main flow	The system validates authorization, generates the export, and makes the result available to the actor.
Alternate flow	If no records match, the system produces an empty result with an explanatory status.
Postconditions	An export artifact exists; source request records are unchanged.

Chapter 11

Architecture Overview

11.1 How to Use This Chapter

Explain the forces that shape the design, the major decisions, alternatives considered, and consequences. Use the 4+1 views only when they help readers answer different architectural questions.

11.2 Decision Template and Example

ARC-001: *[Decision]*; **Drivers:** *[quality attributes and constraints]*; **Rationale:** *[why selected]*; **Consequences:** *[benefits, trade-offs, and risks].* Example: use an asynchronous message queue to protect request processing from transient downstream outages; this improves resilience but introduces eventual consistency.

Chapter 12

User Interface Design

12.1 How to Use This Chapter

Describe personas, primary screens or workflows, interaction rules, feedback, accessibility, and essential states. Include wireframes or prototypes where they resolve meaningful ambiguity.

12.2 Example

Persona: a busy operations coordinator needs to submit and track requests quickly. The request screen groups required fields, identifies inline validation errors, confirms successful submission, supports keyboard operation, and exposes empty, loading, error, and completed states.

Chapter 13

Logical View

13.1 How to Use This Chapter

Model the domain concepts, responsibilities, relationships, data ownership, and invariants without committing to code-level structure. Use a class, entity, or conceptual diagram where useful.

13.2 Example

Request has an identifier, submitter, status, and submitted time. A request has one or more *Request Items*. Invariants: every identifier is unique; every request has a valid status; an approved request cannot return to Draft without an authorized transition.

Chapter 14

Process View

14.1 How to Use This Chapter

Describe runtime behavior, concurrency, state transitions, workflows, error handling, and significant sequences. Use activity, sequence, or state diagrams when prose is insufficient.

14.2 Example State Model

Request states are Draft, Submitted, Under Review, Approved, Rejected, and Cancelled. Submission moves Draft to Submitted only when required data is valid. Approval and rejection are terminal except for an authorized correction workflow.

Chapter 15

Development View

15.1 How to Use This Chapter

Show how source code, modules, services, libraries, and build artifacts are organized. State dependency rules so that implementers do not create forbidden coupling.

15.2 Example

Organize a service into presentation/API, application workflow, domain model, and infrastructure adapters. The API calls application services; application services depend on domain abstractions; infrastructure implements those abstractions. Domain logic must not depend on the web framework or database driver.

Chapter 16

Physical View

16.1 How to Use This Chapter

Describe deployment nodes, networks, environments, runtime dependencies, trust boundaries, data stores, and operations constraints. Include a deployment diagram for distributed systems.

16.2 Example

A browser communicates over TLS with an API service. The API service uses a private database network; backups are encrypted and access is limited to the operations role. Define availability targets, monitoring, and recovery responsibilities.

Chapter 17

Constraints and Assumptions

17.1 Design and Implementation Constraints

Record mandated technologies, budgets, schedules, standards, interfaces, facilities, safety limits, and other restrictions.

17.2 Assumptions and Dependencies

Record assumptions, external dependencies, owners, risks if invalidated, and contingency plans.

Chapter 18

Applicable Codes, Standards, and Regulations

Identify the applicable codes, standards, regulations, policies, and ethical obligations. State how the project will demonstrate compliance.

Chapter 19

Concept Development and Selection

Document alternatives, screening criteria, trade studies, stakeholder input, concept selection rationale, and the resulting baseline concept.

First week of October: present the mission, customer, needs, requirements, and concepts.

Chapter 20

Preliminary and Detailed Design

Document geometry, materials, manufacturing alternatives, software/hardware decomposition, and system-integration plans.

20.1 System Architecture

20.1.1 Overview of System Architecture

Show the system boundary, major subsystems, data/material/energy flows, responsibilities, and integration strategy.

20.1.2 Hardware Architecture

Describe hardware subsystems, interfaces, and allocation of requirements.

20.1.3 Software Architecture

Describe software layers, components, data flows, deployment, and allocation of requirements.

20.1.4 Security Architecture

Describe assets, threats, trust boundaries, controls, access rules, and residual risks as applicable.

20.1.5 Communication Architecture

Describe protocols, networks, message/data formats, timing, and failure handling.

20.2 System Design

20.2.1 System Behavior Description

Describe system states, workflows, use cases, timing, error handling, and safety behavior.

20.2.2 Databases/Data Storage

Describe data models, data lifecycle, retention, backup, privacy, and integrity controls.

20.2.3 Data Conversion

Describe imported, transformed, migrated, or exported data and its validation.

20.2.4 Sub-System Interfaces

Specify interfaces, signals, protocols, APIs, tolerances, and integration responsibilities.

20.2.5 User Interface Design

Present user tasks, wireframes, interaction flows, accessibility considerations, and usability criteria.

20.3 Mechanical Design

20.3.1 Components and Sub-Assemblies - Bill of Materials

Provide the bill of materials, part identifiers, quantities, sources, costs, and assembly relationships.

20.3.2 Geometric Specification - CAD Models/Drawings

Provide or reference controlled CAD models, drawings, dimensions, tolerances, and revision identifiers.

20.3.3 Materials Selected

Justify material selections using relevant mechanical, environmental, cost, and safety criteria.

20.3.4 Manufacturing Processes Selected

Describe selected processes, capabilities, quality controls, and alternatives considered.

20.3.5 Process Description or Optimization Notes

Document process steps, parameters, optimization results, and lessons learned.

20.4 Electronics/Electro-Mechanical Design

20.4.1 Circuits

Provide schematics, circuit rationale, design calculations, and test points.

20.4.2 Electrical and Electro-Mechanical Interfaces

Specify connectors, signals, power, mechanical mounting, protections, and interface constraints.

20.4.3 List of Components: Motors, Sensors, Electronics

List components, part numbers, specifications, source, cost, and selection rationale.

20.4.4 Power System

Describe power sources, budgets, conversion, distribution, protection, battery charging, and safety.

20.5 Software Components Design

20.5.1 Algorithms

Describe algorithms, inputs, outputs, assumptions, complexity/performance, and validation approach.

20.5.2 Modules/Classes/Methods

Describe modules, classes, methods, responsibilities, dependencies, and interfaces.

20.5.3 Application Programming Interfaces

Specify APIs, endpoints, contracts, error behavior, versioning, and authentication as applicable.

20.5.4 Coding/Implementation Details

Describe key implementation decisions, build/run process, configuration, and coding conventions.

Chapter 21

Design Evaluation Methods

21.1 Design Concepts and Concept Screening

Update concept screening if the alternatives or criteria changed after the initial selection.

21.2 Design Evaluation/Selection Methods

Describe the evaluation method, metrics, weights, experimental design, analysis technique, and decision rule.

21.3 Analysis Cases and Results

Present analysis cases, assumptions, calculations, simulation or experiment results, uncertainty, and conclusions.

21.4 Impact on System Design

Explain how analysis results determined system parameters and resulting design revisions.

Third week of November: document the design, finalized concept evaluation and selection, and analysis.

Chapter 22

Alpha Prototyping: Plan and Budget

Define the alpha prototype scope, integration plan, demonstration criteria, budget, procurement, and test plan. The alpha demonstration must show all components and subsystems and how they fit together; full system integration is not required, but every individual subsystem must function.

Chapter 23

Testing and Evaluation: Plan and Results

Provide test objectives, procedures, equipment, data, pass/fail criteria, results, anomalies, corrective actions, and traceability to requirements.

Chapter 24

Tests

24.1 How to Use This Chapter

Define repeatable test cases, levels, environments, data, entry conditions, procedures, expected results, and result recording. Tests produce evidence; they do not replace verification or validation.

24.2 Example Test Cases

Table 24.1: Test Case Examples

ID	Mapped Requirement	Procedure	Expected Result
TC-001	REQ-F-001	Submit a request containing valid required data.	The system creates one request and displays a unique identifier.
TC-002	REQ-F-002	Open the status for a known request.	The current status and most recent update are displayed.
TC-003	REQ-Q-001	Measure 20 status requests under the reference workload.	Every measured response is displayed within two seconds.
TC-004	REQ-F-003	Export a known request set in the approved format.	The export contains the expected history without changing source records.
TC-005	REQ-I-001	Export representative request history and validate the result against the approved interface schema version.	The export conforms to the approved schema and declares the expected schema version.

ID	Mapped Requirement	Procedure	Expected Result
TC-006	REQ-S-001	Attempt the same export as an authorized actor, an authenticated actor without the export role, and an unauthenticated actor.	Only the authorized actor receives an export; both unauthorized attempts are denied and recorded.
TC-007	REQ-D-001	Perform two status transitions and inspect the stored history and exported record.	Each transition retains the expected timestamp, actor identifier, previous status, and new status.
TC-008	REQ-C-001	Execute the critical use-case test suite on every platform and browser version in the approved deployment baseline.	Every critical use case passes on every approved platform and browser version.
TC-009	REQ-B-001	Time an authorized analyst preparing an approved export from the reference data set without manual reconstruction.	The analyst obtains analysis-ready request-history data within five minutes.

Chapter 25

Traceability

25.1 How to Use This Chapter

Maintain the canonical mapping from stakeholder need or approved constraint through requirement, design decision, implementation, test, verification, validation, and release evidence. Review both missing backward links and missing forward links.

25.2 Traceability Register

Requirement and use-case chapters shall contain reciprocal mappings. An unlinked requirement may be gold-plating; an untested requirement or unmapped use case is incomplete.

Table 25.1: Traceability Register

User Story	Requirement	Use Case	Test Case	Verification	Validation
US-001	REQ-F-001	UC-001	TC-001	VER-001	VAL-001
US-002	REQ-F-002	UC-002	TC-002	VER-002	VAL-002
US-002	REQ-Q-001	UC-002	TC-003	VER-003	VAL-002
US-003	REQ-F-003	UC-003	TC-004	VER-004	VAL-003
US-003	REQ-I-001	UC-003	TC-005	VER-005	VAL-003
US-003	REQ-S-001	UC-003	TC-006	VER-006	VAL-003
US-002	REQ-D-001	UC-002	TC-007	VER-007	VAL-002
US-003	REQ-D-001	UC-003	TC-007	VER-007	VAL-003
US-001	REQ-C-001	UC-001	TC-008	VER-008	VAL-001
US-002	REQ-C-001	UC-002	TC-008	VER-008	VAL-002
US-003	REQ-C-001	UC-003	TC-008	VER-008	VAL-003
US-003	REQ-B-001	UC-003	TC-009	VER-009	VAL-003

Chapter 26

Verification

26.1 How to Use This Chapter

Show that each requirement conforms to its specification using test, inspection, analysis, or demonstration. Record the method, evidence reference, responsible reviewer, result, and any waiver. Verification answers "Did we build the system right?"

26.2 Verification Register

Table 26.1: Verification Register

ID	Requirement	Method	Required Evidence	Status
VER-001	REQ-F-001	Test	Passing TC-001 execution record showing creation of one request with a unique identifier.	**PASS**
VER-002	REQ-F-002	Test	Passing TC-002 execution record showing the current status and most recent update.	**PASS**
VER-003	REQ-Q-001	Analysis and test	Timing analysis and passing TC-003 measurements under the reference workload.	**PASS**
VER-004	REQ-F-003	Inspection and test	Export inspection and passing TC-004 execution record.	**PENDING**
VER-005	REQ-I-001	Inspection and test	Schema-validation report and passing TC-005 execution record.	**PENDING**
VER-006	REQ-S-001	Analysis and test	Access-control analysis, denial records, and passing TC-006 execution record.	**PASS**
VER-007	REQ-D-001	Inspection and test	Stored-history inspection and passing TC-007 execution record.	**PASS**

Continued on next page

Verification Register (continued)

ID	Requirement	Method	Required Evidence	Status
VER-008	REQ-C-001	Test	Approved deployment baseline and passing TC-008 compatibility results.	**PASS**
VER-009	REQ-B-001	Test	Timed analyst demonstration and passing TC-009 execution record.	**PENDING**

Chapter 27

Validation

27.1 How to Use This Chapter

Evaluate whether the completed system solves stakeholder needs in its intended context. Define realistic scenarios, participant roles, success criteria, observations, and disposition of requested changes. Validation answers "Did we build the right system?"

27.2 Validation Register

Table 27.1: Validation Register

ID	Related Story	Scenario	Success Criterion	Status
VAL-001	US-001	A customer submits a realistic service request without coaching.	The customer receives a unique identifier and confirms that the workflow supports the intended task.	PENDING
VAL-002	US-002	A customer checks the progress of an existing request.	The customer correctly identifies the current status and latest update and judges the response timely.	PENDING

Continued on next page

Validation Register (continued)

ID	Related Story	Scenario	Success Criterion	Status
VAL-003	US-003	An authorized analyst exports a representative request history.	Within five minutes, the analyst opens the approved format, finds the expected records, and confirms source records were unchanged.	**PENDING**

Third week of February.

Chapter 28

Beta Prototyping: Design Revision and Optimization

28.1 Mechanical Design

Describe mechanical revisions and optimizations made after alpha testing.

28.2 Electronics/Electro-Mechanical Design

Describe electrical and electro-mechanical revisions and optimizations made after alpha testing.

28.3 Software Design

Describe software revisions, defect corrections, performance improvements, and configuration changes.

28.4 System-Level Design

Describe integration changes, system-level optimization, and readiness for beta demonstration.

Chapter 29

Fabrication, Assembly, and Manufacturing Processes

Document fabrication, assembly, manufacturing, quality-control steps, process outcomes, and lessons learned.

Chapter 30

High-Performance Testing and Evaluation

Present final integrated-system tests under realistic user environments, results, limitations, and evidence that requirements are met.

Last week of March: the design and prototype must be reasonably complete. The beta demonstration must integrate all subsystems into a fully functional system and demonstrate operation in realistic user environments.

Chapter 31

Final Design Specification

Provide the final controlled specification, final configuration, delivered design documents, requirements status, test evidence, known limitations, and acceptance status. Include detailed design documents in Appendix D.

Spring finals week.

Part III

Entrepreneurship and Business Development

Chapter 32

Business Objectives and Risks

Define the value proposition, target outcomes, business objectives, major risks, mitigations, and evidence used to assess feasibility.

Chapter 33

Competitive Intelligence: Market Analysis

Analyze target users, market segments, alternatives, competitors, differentiation, market constraints, and sources.

Chapter 34

Lean Canvas Business Model

Present the problem, customer segments, unique value proposition, solution, channels, revenue streams, cost structure, key metrics, and unfair advantage.

Chapter 35

Financial Analysis

Document costs, pricing or funding assumptions, revenue/value model, break-even analysis, financial risks, and sensitivity analysis.

Chapter 36

Intellectual Property

Document ownership, prior art, licensing, patents, trademarks, copyrights, open-source obligations, and confidentiality considerations.

Chapter 37

Innovation Expo

Describe pitch presentations, innovation showcase demonstrations, audience, materials, feedback, and resulting actions.

Part IV

Appendices

Appendix A

Team Organization Chart

Insert the current organization chart, role definitions, responsibility assignments, and advisor/sponsor relationships.

Appendix B

Project Gantt Chart

Insert the current Gantt chart, critical path, dependencies, milestone dates, and status.

Appendix C

Prototyping and Testing Budget

Provide planned and actual prototype/testing costs, funding source, procurement status, and variance explanations.

Appendix D

Design Documents: Drawings, Layouts, Analysis Reports

Include or reference revision-controlled drawings, layouts, CAD models, schematics, analysis reports, calculations, and other detailed design artifacts.

Appendix E

References Cited

List references cited in the report. Maintain complete bibliographic records in the bibliography database; the generated bibliography follows the appendices.

Appendix F

Document Authoring Guide

F.1 How to Use This Appendix

Use this appendix to maintain authoring conventions, repository locations, diagram sources, review rules, tool installation, and PDF build instructions. Update it whenever the document structure or toolchain changes.

F.2 Template Provenance

This template was developed by **Darian Muresan, Ph.D.**, in conjunction with the AI agents **OpenAI Codex** and **Anthropic Claude Code**. Questions, corrections, and proposed improvements may be sent to dmuresan@stevens.edu, darian@dmmd.net, or darian@pictomic.com.

Keep the template attribution distinct from the project-specific document owner and organization. A derived project may replace the project metadata while retaining this provenance statement for the underlying template.

F.3 AI-Assisted Authoring

The authoritative instructions for AI-assisted architecture work, including task routing, limited context loading, workflows, example prompts, completion checks, and human-review boundaries, are contained in Chapter G. This appendix retains installation, build, file, and LaTeX authoring guidance that applies to both human authors and AI agents.

F.4 File Naming

Every chapter file uses the lowercase namespace `dsn`, followed by a Pascal-style responsibility name and the `.tex` extension. Examples are `dsnRequirements.tex`, `dsnTests.tex`, and `dsnValidation.tex`. Supporting diagram files use the same prefix.

F.5 One Responsibility per Chapter

A chapter answers one major engineering question. When content belongs to another responsibility, link to its authoritative chapter instead of copying it. In particular, tests define procedures, verification records requirement compliance, validation evaluates stakeholder use, and release decides whether the evidence is sufficient.

F.6 Adding a Chapter

Create the namespaced file, begin it with a chapter title and stable chapter label, add useful index terms, and include it once from `dsnManual.tex`. Add or change artifact mappings only in the canonical traceability matrix.

```
\chapter{Example Chapter}
\ChapterLabel{ExampleChapter}
\index{example}
\index{Chapter!Example Chapter}

Chapter content goes here.
```

F.7 Building the PDF

The build files regenerate all Graphviz diagrams before compiling the document. They require:

- Graphviz, including the `dot` command;
- a LaTeX distribution containing `pdflatex`, `bibtex`, and `latexmk`; and
- the `makeglossaries` and `makeindex` commands.

F.7.1 Installing the Tools on macOS

The full MacTeX distribution is recommended because it includes the LaTeX tools and packages used by this template. Complete these steps from an administrator account:

1. Open Terminal and install Apple's command-line tools, which provide `make`:

   ```
   xcode-select --install
   ```

   Complete the installer dialog before continuing.

2. Install Homebrew by following the current instructions at https://brew.sh/. Then install Graphviz and MacTeX:

```
brew install graphviz
brew install --cask mactex
```

MacTeX may instead be downloaded directly from https://tug.org/mactex/.

3. Close and reopen Terminal so the MacTeX command-line tools are added to the system path.

4. Verify that every required command is available:

```
command -v make dot pdflatex bibtex \
  makeglossaries makeindex latexmk
```

Each tool shall print a path. A missing line identifies the tool that still needs to be installed or added to the system path.

F.7.2 Installing the Tools on Windows

Use a current Windows 10 or Windows 11 computer and complete these steps:

1. Download and run the Basic MiKTeX Installer from https://miktex.org/howto/install-miktex. Choose a private per-user installation when appropriate and select *Always install missing packages on-the-fly* for a computer with Internet access.

2. Open MiKTeX Console, check for updates, and install them. If `latexmk` is not found during verification, use the Packages page in MiKTeX Console to install the `latexmk` package.

3. Open PowerShell and install Graphviz with Windows Package Manager:

```
winget install graphviz
```

If `winget` is unavailable, use the current 64-bit installer from https://graphviz.org/download/ and enable the option that adds Graphviz to the system path.

4. Close and reopen PowerShell, then verify the required commands:

```
Get-Command dot,pdflatex,bibtex
Get-Command makeglossaries,makeindex,latexmk
```

Each tool shall produce a command record. A command-not-found error identifies a missing installation or system-path entry.

On an organization-managed computer, authors shall ask the system administrator before installing software or changing a managed execution policy.

F.7.3 Installing the Tools on Ubuntu or Debian Linux

Open a terminal and install Graphviz, GNU Make, Latexmk, and a TeX Live set that contains the packages used by this template:

```
sudo apt update
sudo apt install graphviz make latexmk texlive-latex-extra \
  texlive-fonts-recommended texlive-bibtex-extra \
  texlive-extra-utils
```

Verify the required commands:

```
command -v make dot pdflatex bibtex \
  makeglossaries makeindex latexmk
```

For another Linux distribution, install the equivalent Graphviz, GNU Make, Latexmk, and full or extended TeX Live packages with its package manager.

F.7.4 AI-Agent Installation

An AI coding agent can inspect the host, identify missing commands, request approval for system changes, install the appropriate packages, and verify the build. Give the agent the project directory and the following reusable request:

```
In this LaTeX project, inspect the build files and determine
which operating system and package manager are available.
Verify make, dot, pdflatex, bibtex, makeglossaries,
makeindex, and latexmk. Install only missing tools using the
platform's standard package manager. Ask for approval before
administrator-level, system-wide, or networked installation.
Do not replace project sources or remove user files. After
installation, regenerate the Graphviz diagrams, run the full
platform build, inspect the log for errors, unresolved
references, and overfull boxes, and report the generated PDF.
```

If the agent cannot install software, ask it to report the exact missing command, the attempted verification, and platform-specific installation instructions. Never give an agent an unrestricted instruction to delete build directories or modify system security policy.

F.7.5 Building on macOS and Linux

From the template directory, run:

```
make -f makefileMacLinux pdf
```

Use `make -f makefileMacLinux diagrams` to regenerate only the diagrams and `make -f makefileMacLinux clean` to remove generated build files while preserving authored sources and the final PDF.

F.7.6 Building with Windows PowerShell

From PowerShell in the template directory, run:

```
.\makefilePowerShell.ps1 -Action pdf
```

Use `-Action diagrams` or `-Action clean` in place of `-Action pdf` to run those operations independently. If local execution policy prevents scripts from running, use an approved organizational policy or allow scripts only for the current PowerShell process:

```
Set-ExecutionPolicy -Scope Process Bypass
.\makefilePowerShell.ps1 -Action pdf
```

F.7.7 Manual Build Sequence

When the platform build script is unavailable, run the following commands from the template directory. The final two LaTeX passes resolve tables of contents, citations, glossary links, the index, and cross-references.

```
dot -Gdpi=180 -Tpng diagrams/dsnVModel.dot \
  -o png/dsnVModel.png
dot -Gdpi=180 -Tpng diagrams/dsnUseCaseDiagram.dot \
  -o png/dsnUseCaseDiagram.png
pdflatex -interaction=nonstopmode -halt-on-error dsnManual.tex
bibtex dsnManual
makeglossaries dsnManual
makeindex dsnManual.idx
pdflatex -interaction=nonstopmode -halt-on-error dsnManual.tex
pdflatex -interaction=nonstopmode -halt-on-error dsnManual.tex
```

The provided build scripts regenerate every `.dot` diagram, not only the two shown in this abbreviated manual sequence.

F.7.8 Build Verification and Troubleshooting

Confirm that the output PDF opens, all expected chapters and appendices appear, the glossary and index are populated, hyperlinks reach their targets, and no table or figure is clipped. Search `dsnManual.log` for fatal errors, undefined references, missing files, and overfull boxes. Common remedies are:

- open a new terminal after installation so its search path is refreshed;
- on macOS, verify that `/Library/TeX/texbin` is on the search path;
- rerun the complete build after adding citations, glossary entries, index entries, figures, labels, or references;
- regenerate diagrams after changing a `.dot` source; and
- install the named LaTeX package when the log reports a missing `.sty` file.

F.8 Authoring Rules

- Add the newest approved change to the top of the document history.
- Introduce specialized terms through the glossary and use them consistently.
- Give every controlled artifact a stable explicit ID.
- Use `shall` only for binding requirements.
- Keep normative requirements separate from explanation and examples.
- Store editable diagram sources in `diagrams/` and rendered images in `png/`.
- Compile and inspect the PDF after structural or table changes.

F.9 Citations and Bibliography

Store controlled reference records in `dsnReferences.bib`. Cite a source from a chapter with a stable BibTeX key:

```
\cite{ISO29148}
```

The main file applies the style and prints one document-level bibliography:

```
\bibliographystyle{IEEEtran}
\bibliography{dsnReferences}
```

Both platform build scripts run BibTeX automatically.

Bibliography entries support claims and definitions; they do not replace project-owned requirements, architecture decisions, test procedures, or approval evidence.

F.10 Selective Builds

For a large project, LaTeX's `\includeonly` command may select chapters during drafting. The main file contains a commented example:

```
% \includeonly{
%   prologue,
%   dsnRequirements,
%   dsnTests
% }
```

Remove the leading comment characters and list include names without the `.tex` extension. LaTeX can retain cross-references and citations from omitted chapters by reading their existing auxiliary files, but those files may be absent or stale. Selective builds are therefore drafting aids only. Hyperref may also report missing destinations for links into deliberately omitted chapters; these draft warnings shall disappear in the full build. Comment out the complete `\includeonly` block and run a full build before verification or release so missing chapters, references, glossary entries, citations, and evidence cannot be hidden.

Appendix G

AI Agent Usage Guide

G.1 How to Use This Appendix

This appendix is the entry point for an AI agent that creates, reviews, or maintains this architecture specification. The agent shall read this file first, identify the requested task in the task-routing table, and load only the authoritative sources listed for that task. It shall expand the context when a referenced dependency, cross-cutting decision, or inconsistency requires it.

This appendix replaces separate AI-oriented Markdown instructions. Its routing tables, context packs, workflows, prompts, and checklists are maintained as LaTeX so they remain visible in the architecture book and versioned with the specification.

G.2 Governing Principles

The authored LaTeX files and editable diagram sources are the source of truth. The generated PDF is a review artifact and shall not be edited directly. The agent shall preserve one authoritative location for each fact and use links instead of duplicating content across chapters.

An AI agent is a drafting, analysis, consistency-checking, and build assistant. It shall use supplied project evidence, identify missing information, expose material assumptions, and distinguish proposals from approved decisions. It shall not invent requirements approval, implementation status, test results, verification evidence, validation evidence, waivers, or release authorization.

G.3 AI Quick Start

1. Read this appendix and identify the requested outcome.
2. Select the closest entry in the task-routing table.
3. Read the listed primary files and the applicable context pack. Read `dsnManual.tex` when document metadata, macros, inclusion order, or build structure is affected.

4. Inspect available project evidence and report conflicts, missing facts, and consequential assumptions. Ask for a human decision when an answer would materially change scope, architecture, safety, security, cost, schedule, or release readiness.

5. Make narrowly scoped changes in the authoritative files. Preserve stable IDs and update every affected reciprocal mapping.

6. Add a linked document-history entry for an approved document change and record material AI interactions in Chapter H.

7. Run the complete build, inspect the log, render affected PDF pages, and complete the completion checklist.

G.4 Task Routing

The file lists below are minimum starting contexts, not permission to ignore a dependency discovered during the task. A task that changes an artifact shall also inspect every register that contains a reciprocal mapping to that artifact.

Table G.1: AI Task-Routing Table

Task	Minimum Authoritative Sources
Initialize a project	`dsnManual.tex`, `prologue.tex`, `dsnIntroduction.tex`, `dsnDevelopmentPlan.tex`, `dsnSystemContext.tex`, and the supplied project evidence.
Develop user stories	`dsnSystemContext.tex`, `dsnUserStories.tex`, `dsnRequirements.tex`, and stakeholder evidence.
Add or change requirements	`dsnUserStories.tex`, `dsnRequirements.tex`, `dsnUseCases.tex`, `dsnTests.tex`, `dsnTraceability.tex`, `dsnVerification.tex`, and `dsnGlossary.tex`.
Add or change use cases	`dsnUserStories.tex`, `dsnRequirements.tex`, `dsnUseCases.tex`, `dsnTests.tex`, and `dsnTraceability.tex`.
Develop or change architecture	`dsnSystemContext.tex`, `dsnRequirements.tex`, `dsnArchitectureOverview.tex`, and the affected interface, logical, process, development, physical, or implementation chapters and diagrams.
Add or change tests	`dsnRequirements.tex`, `dsnUseCases.tex`, `dsnTests.tex`, `dsnTraceability.tex`, `dsnVerification.tex`, and `dsnTestEvidence.tex`.
Perform verification	`dsnRequirements.tex`, `dsnTests.tex`, `dsnTraceability.tex`, `dsnVerification.tex`, and controlled test evidence.
Perform validation	`dsnUserStories.tex`, `dsnUseCases.tex`, `dsnValidation.tex`, and controlled stakeholder evidence.

Continued on next page

AI Task-Routing Table (continued)

Task	Minimum Authoritative Sources
Prepare a release	`dsnRequirements.tex`, `dsnTraceability.tex`, `dsnVerification.tex`, `dsnValidation.tex`, `dsnRelease.tex`, and controlled release evidence.
Review consistency	All registers containing the affected artifact types, `dsnGlossary.tex`, `dsnDocumentHistory.tex`, and `dsnManual.tex`.
Compile or troubleshoot	`dsnManual.tex`, `dsnDocumentGuide.tex`, the affected chapter, its diagram sources, and the build log.

G.5 Task-Specific Context Packs

A context pack is a reading boundary within this appendix, not a separate file. Use the applicable pack together with the task-routing entry.

G.5.1 Requirements Context

Establish the system boundary and stakeholder need before proposing a requirement. Keep each requirement singular, feasible, unambiguous, and verifiable. Classify its primary family or subtype, write binding behavior with **shall**, assign one Must, Should, Could, or Would priority, assign one lifecycle status, and map it to at least one use case and one test case. A requirement may be marked Done only when its mapped tests pass, verification is complete, traceability is current, and a controlled implementation version is known.

G.5.2 Use-Case Context

Start from an actor goal and a system boundary. Give each use case its own table, identify participating actors, preconditions, trigger, success outcome, main flow, alternate and exception flows, postconditions, mapped requirements, and mapped tests. Describe externally observable interaction rather than internal implementation detail. Maintain reciprocal mappings to every related requirement.

G.5.3 Architecture Context

Trace each architectural decision to requirements, quality attributes, constraints, risks, and interfaces. Identify alternatives and explain the trade-off that selected the proposed structure. Keep the context, logical, process, development, physical, interface, and implementation views consistent. Update editable diagram sources and regenerate their rendered images.

G.5.4 Testing and Evidence Context

Derive tests from requirements and use cases. Define preconditions, inputs, procedure, expected result, and retained evidence. A test definition is not a test result. Verification records objective compliance with requirements; validation records whether stakeholder needs are satisfied in intended use. Never create passing evidence or approval from an expected result.

G.5.5 Release Context

Review open, in-progress, done, and retired requirements; unresolved defects; verification and validation results; operational readiness; migration and rollback provisions; known limitations; and required approvals. Report a gap instead of treating missing evidence as acceptance. Only an authorized human may approve release.

G.5.6 Document-Authoring Context

Follow Chapter F for file naming, LaTeX conventions, diagrams, installation, and builds. Update the glossary and index when introducing a controlled term. Update document history with a link to the changed location. Do not edit generated auxiliary files or the PDF directly.

G.6 Mandatory Consistency Rules

1. Preserve every stable ID. Do not reuse an ID belonging to a retired or deleted proposal.
2. Use **shall** for binding requirements. Use priority terms only in the Priority field and with the definitions in the glossary.
3. Give every requirement a priority, status, In Version value, use-case mapping, and test-case mapping.
4. Enter ∅ when a requirement is not Done and no completed implementation version applies.
5. Maintain reciprocal mappings among user stories, requirements, use cases, tests, traceability, verification, and validation.
6. Do not copy one controlled statement into several authoritative locations. Link to its stable ID.
7. Retain retired requirements and their history. Record the rationale and replacement when applicable.
8. Use project evidence for factual claims. Label an unsupported suggestion as a proposal and report the evidence needed to decide it.
9. Link every document-history entry to the location where the change occurred.

10. Compile after structural, cross-reference, glossary, index, diagram, or table changes and visually inspect the affected pages.

G.7 Standard AI Workflows

G.7.1 Initialize a Project

Replace project metadata and bracketed placeholders only from supplied facts. Create a gap list covering purpose, scope, stakeholders, system boundary, interfaces, constraints, quality attributes, risks, evidence, and approvals. Resolve consequential gaps before developing detailed artifacts. Work in dependency order from context and needs through requirements, use cases, architecture, tests, evidence, and release.

G.7.2 Add or Change a Requirement

Identify the originating need and affected use cases. Draft a singular verifiable obligation, select its primary category, priority, and status, and assign a stable ID. Add or update mapped use cases and tests, then update traceability and verification. Confirm every link in both directions and add a linked history entry.

G.7.3 Add or Change a Use Case

Identify the actor goal, system boundary, and mapped requirements. Complete one use-case table with success, alternate, and exception behavior. Add mapped tests, update requirement mappings and traceability, and confirm that no flow introduces an unrecorded requirement.

G.7.4 Develop or Change Architecture

Identify the requirements and quality attributes that drive the decision. Describe alternatives, trade-offs, risks, interfaces, and the selected structure. Update every affected architectural view and diagram. Confirm that the proposed design neither omits a driving requirement nor silently adds an unapproved constraint.

G.7.5 Add Tests or Evidence

Derive each test from mapped requirements and use-case flows. Define repeatable steps and objective expected results. Link retained evidence only after an actual execution. Update verification or validation records without converting an expected outcome into a claimed result.

G.7.6 Review and Prepare a Release

Audit IDs, mappings, statuses, versions, unresolved assumptions, verification, validation, operational readiness, and release criteria. Report contradictions and missing evidence.

Prepare a release recommendation for human approval; do not approve the release.

G.8 Example AI Requests and Prompts

Replace bracketed text with project facts. A prompt does not override this guide, the controlled project evidence, or human approval boundaries.

G.8.1 Initialize the Architecture Book

```
Read dsnAIAgentGuide.tex and use the Initialize a Project workflow
for [Project Name]. Review the supplied project evidence and create
a gap list before editing. Identify missing scope, stakeholders,
interfaces, constraints, quality attributes, risks, evidence, and
approvals. After the gap list is reviewed, replace only supported
placeholders and develop the document in dependency order. Compile
the complete PDF and report unresolved human decisions.
```

G.8.2 Develop Requirements

```
Read dsnAIAgentGuide.tex and follow the Requirements context and
workflow. From the approved objectives and user stories, propose
singular, feasible, and verifiable requirements. Classify each one,
use shall for the binding obligation, assign one priority and one
status, and map it to at least one use case and test case. Do not
mark work Done or assign an implementation version without evidence.
Update reciprocal mappings, history, and the index; then build and
inspect the affected pages.
```

G.8.3 Develop Use Cases

```
Read dsnAIAgentGuide.tex and follow the Use-Case context and
workflow. Derive use cases from the approved actor goals and mapped
requirements. Give each use case its own table with actors, trigger,
preconditions, main flow, alternate and exception flows, outcome,
postconditions, requirements, and tests. Maintain reciprocal links
and report any flow that requires a new requirement. Build and
inspect the affected pages.
```

G.8.4 Develop an Architecture Decision

```
Read dsnAIAgentGuide.tex and follow the Architecture context and
workflow for [decision or subsystem]. Identify driving requirements,
quality attributes, constraints, interfaces, and risks. Compare
reasonable alternatives and document the trade-off supporting the
proposed choice. Update every affected architecture view and editable
diagram source. Do not present the proposal as approved. Compile the
document and inspect all affected diagrams and pages.
```

G.8.5 Develop Tests and Traceability

```
Read dsnAIAgentGuide.tex and follow the Testing and Evidence context.
For [requirements or use cases], define repeatable tests with inputs,
preconditions, steps, expected results, and required evidence. Update
all reciprocal mappings and the traceability matrix. Do not invent an
execution result or mark verification complete. Compile the document,
check references, and inspect the affected tables.
```

G.8.6 Review Release Readiness

```
Read dsnAIAgentGuide.tex and follow the Release context and workflow
for [release]. Audit requirement status and implementation versions,
traceability, verification, validation, defects, operations, rollback,
known limitations, and approvals. Separate confirmed evidence from
gaps and assumptions. Prepare a release-readiness recommendation for
human review, but do not approve release. Build and inspect the final
document.
```

G.9 Human Decisions and Information Boundaries

An authorized human shall approve scope, requirements, architecture decisions, security and privacy posture, safety decisions, evidence sufficiency, waivers, validation conclusions, and release. The agent shall present alternatives and evidence without claiming that review has occurred.

Do not provide an AI agent with credentials, secrets, personal data, production data, export-controlled material, or other information it is not authorized to process. Before using external sources, confirm that the task permits network access and that the source may be incorporated. Record material prompts, assumptions, corrections, and unresolved issues in Chapter H.

G.10 Completion Checklist

Table G.2: AI-Assisted Change Completion Checklist

Area	**Required Completion Evidence**
Scope	The requested outcome is complete; unrelated chapters and user changes remain untouched.
Evidence	New factual statements are supported by supplied or controlled evidence; proposals, assumptions, and unknowns are identified.
Language	Binding requirements use shall; priority and status use the controlled terms and definitions.
Identifiers	Stable IDs remain unique and are neither renumbered nor reused.
Traceability	Every changed requirement, use case, test, verification record, and validation record has complete reciprocal mappings.
History	The newest approved document change appears at the top of document history and links to the changed location.
Glossary and index	New controlled terms and important concepts are defined and indexed where applicable.
Build	The complete platform build succeeds; citations, glossary, index, diagrams, and cross-references are regenerated.
Log review	No fatal error, undefined reference, duplicate label, missing file, or unresolved layout warning remains.
Visual review	Affected PDF pages show readable tables, correct page breaks, current diagrams, working headings, and no clipped or overlapping content.
Human review	Decisions requiring authority are clearly identified and are not represented as approved without evidence.

Appendix H

AI Prompt Log

H.1 How to Use This Appendix

Use this log when AI assistance materially affects requirements, architecture, code, tests, or release evidence. Record enough context for a reviewer to understand the input, output, assumptions, and human corrections.

H.2 Example Record

Date: *[timestamp]*; Tool and model: *[name]*; Input: *[requirements and constraints supplied]*; Task: *[requested output]*; Human review and corrections: *[summary]*; Unresolved assumptions: *[list or none]*.

Bibliography

[1] *ISO/IEC/IEEE 15288:2023, Systems and Software Engineering—System Life Cycle Processes*, International Organization for Standardization Std. ISO/IEC/IEEE 15 288:2023, 2023. [Online]. Available: https://www.iso.org/standard/81702.html

[2] *ISO/IEC/IEEE 12207:2026, Systems and Software Engineering—Software Life Cycle Processes*, International Organization for Standardization Std. ISO/IEC/IEEE 12 207:2026, 2026. [Online]. Available: https://www.iso.org/standard/90219.html

[3] *ISO/IEC/IEEE 29148:2018, Systems and Software Engineering—Life Cycle Processes—Requirements Engineering*, International Organization for Standardization Std. ISO/IEC/IEEE 29 148:2018, 2018. [Online]. Available: https://www.iso.org/standard/72089.html

Glossary

acceptance criterion An observable condition used to decide whether a need, story, or requirement has been satisfied. 85

Could A desirable capability included only when time and resources permit. 2, 21, 78, 85

Must A capability required in the current release; the release is unacceptable without it. 2, 20–24, 78, 85

requirement A necessary, singular, unambiguous, feasible, and verifiable statement of capability or constraint. 85

Should An important capability expected in the current release unless a documented constraint prevents it. 2, 21, 22, 24, 78, 85

stakeholder A person, group, or organization that affects the system or is affected by it. 85

traceability The ability to follow an artifact backward to its source and forward to its implementation and evidence. 85

use case A goal-oriented interaction between an actor and the system, including normal and alternate flows. 85

V-model A lifecycle model that pairs each decomposition and specification activity with a corresponding integration and evidence activity. 3, 85

validation Confirmation that the completed system satisfies stakeholder needs in its intended context; building the right system. 85

verification Confirmation that specified requirements have been fulfilled; building the system right. 85

Would A capability assigned to a later implementation version after higher-priority work. 2, 21, 24, 78, 85

Acronyms

AI artificial intelligence. 1, 85

UI user interface. 85

UML Unified Modeling Language. 85

V&V verification and validation. 85

Index

acceptance criterion, 24
AI agent, 76
  architecture workflow, 2, 70, 80
  completion checklist, 82
  consistency rules, 79
  context pack, 78
  context routing, 76
  example prompts, 76, 81
  human review, 76, 82
  LaTeX installation prompt, 70
  quick start, 76
  task routing, 77
Appendix
  AI Agent Usage Guide, 76
  Document Authoring Guide, 70

build tools, 70

Chapter
  Introduction, 1

document conventions, 70
document history, v

introduction, 1

LaTeX, 70

Muresan, Darian, iii

normative language, 1

priority, 1, 14

Requirement
  REQ-B-001, 20
  REQ-C-001, 23
  REQ-D-001, 23
  REQ-F-001, 21
  REQ-F-002, 21
  REQ-F-003, 21
  REQ-I-001, 22
  REQ-Q-001, 21
  REQ-S-001, 22
requirement, 14
  business, 20
  category catalog, 15
  constraint, 23
  data, 23
  functional, 14
  implementation version, 14
  interface, 22
  quality, 14
  security, 22
  status, 15
requirements traceability matrix, 46

source of truth, 1, 76
status
  Done, 15
  In Progress, 15
  Open, 15
  Retired, 15

template attribution, iii
template provenance, 70
TestCase
  TC-001, 44
  TC-002, 44
  TC-003, 44
  TC-004, 44
  TC-005, 44
  TC-006, 45
  TC-007, 45
  TC-008, 45
  TC-009, 45
traceability, 46

use case diagram, 26
UseCase
    UC-001, 26
    UC-002, 27
    UC-003, 27
user story, 24
UserStory
    US-001, 24
    US-002, 24
    US-003, 24

V-model, 1
Validation
    VAL-001, 50
    VAL-002, 50
    VAL-003, 51
validation, 50
validation scenario, 50
Verification
    VER-001, 48
    VER-002, 48
    VER-003, 48
    VER-004, 48
    VER-005, 48
    VER-006, 48
    VER-007, 48
    VER-008, 49
    VER-009, 49
verification, 48
verification method, 48